SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I SARU HASINI.S (192521418) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SARU HASINI.S

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

A. Identifying and Fixing Logical and Implementation Errors

A DES/3DES implementation producing different ciphertext for the same plaintext and key usually indicates **non-deterministic behavior caused by incorrect key handling, permutation logic, or state management**. DES is deterministic: the same key and plaintext must always generate the same ciphertext.

1. Key Scheduling Errors

DES uses a **16-round Feistel structure**. Each round requires a unique 48-bit subkey generated from the original 64-bit key.

Common key scheduling mistakes:

Error	Effect	Fix
Incorrect PC-1 permutation	Wrong 56-bit effective key	Verify PC-1 table and bit indexing
Mixing 0-based and 1-based indexing	Incorrect subkeys	Convert DES tables carefully to Python indexing
Incorrect left shifts	Different round keys	Use DES-defined shift schedule
Generating new random keys internally	Different ciphertext each run	Use fixed input key
Modifying the original key during rounds	Corrupted key schedule	Preserve original key and generate independent subkeys

Correct DES key schedule:

64-bit Key
|
↓
PC-1 Permutation
|
56-bit Key
|
Split into C0 and D0 (28 bits each)
|
Left circular shifts
|
PC-2 Permutation
|

48-bit Round Key
Example correction:

```
def left_shift(bits, n):
    return bits[n:] + bits[:n]


def generate_keys(key):
    key = permute(key, PC1)

    C, D = key[:28], key[28:]

    round_keys = []

    shifts = [
        1,1,2,2,2,2,2,2,
        1,2,2,2,2,2,2,1
    ]

    for shift in shifts:
        C = left_shift(C, shift)
        D = left_shift(D, shift)

        combined = C + D
        round_key = permute(combined, PC2)

        round_keys.append(round_key)

    return round_keys
```

---

2. Permutation Errors

DES depends heavily on fixed permutations:

- Initial Permutation (IP)
- Expansion permutation (E)
- Permuted Choice 1 (PC-1)
- Permuted Choice 2 (PC-2)
- P-box permutation
- Final permutation (IP^{-1})

Common problems:

Incorrect table indexing

DES tables use positions starting from **1**, while Python lists use **0**.

Incorrect:

```
output.append(bits[index])
```

Correct:

```
output.append(bits[index-1])
```

Incorrect final permutation

DES encryption flow:

```
Plaintext
  |
Initial Permutation (IP)
  |
16 Feistel rounds
  |
Swap L16 and R16
  |
```

Inverse Initial Permutation
  |
Ciphertext
The final swap is frequently implemented incorrectly.
Correct:
combined = right + left
ciphertext = permute(combined, IP_inverse)

---

3. S-Box Implementation Errors
Typical problems:
- Wrong row calculation
- Wrong column calculation
- Incorrect binary conversion

Correct S-box indexing:
row = int(block[0] + block[5], 2)
column = int(block[1:5], 2)

value = S_BOX[i][row][column]

---

4. 3DES Implementation Errors
3DES performs:

$$C = E_{K3}\left(D_{K2}\left(E_{K1}(P)\right)\right)$$

Encryption:
cipher = DES_encrypt(plaintext, K1)
cipher = DES_decrypt(cipher, K2)
cipher = DES_encrypt(cipher, K3)
Decryption:
plain = DES_decrypt(ciphertext, K3)
plain = DES_encrypt(plain, K2)
plain = DES_decrypt(plain, K1)
Common errors:
- Using encryption for all three stages
- Incorrect key order
- Reusing mutable DES state between stages

B. Optimization for Large Inputs (≥10 MB)
A pure Python DES implementation becomes slow because DES performs:
- 16 rounds per block
- Multiple permutations
- Bit-level operations

A 10 MB file contains:

$$\frac{10 \times 1024 \times 1024}{8} = 1{,}310{,}720$$

DES blocks, requiring millions of operations.
Optimization Techniques
1. Precompute Permutation Tables
Instead of recalculating permutations:
Before:
for i in table:
  result += bits[i-1]
Optimized:
IP_TABLE = tuple(x-1 for x in IP)

```
def fast_permute(bits):
    return ''.join(bits[i] for i in IP_TABLE)
```

---

2. Use Integer Bit Operations

Strings are slow for cryptography.

Instead of:

```
"10101010"
```

use:

```
0b10101010
```

Advantages:

- Faster XOR
- Faster shifts
- Lower memory usage

Example:

```
left ^= right
```

is faster than:

```
''.join(str(int(a)^int(b)))
```

---

3. Cache Round Keys

Incorrect:

```
for block in data:
    keys = generate_keys(key)
```

Optimized:

```
keys = generate_keys(key)

for block in data:
    encrypt(block, keys)
```

This avoids generating 16 keys for every block.

---

4. Process Data in Blocks

Do not load the entire file into memory.

Use streaming:

```
with open("input.bin","rb") as f:
    while block := f.read(8):
        encrypt(block)
```

Benefits:

- Lower memory consumption
- Better scalability

---

5. Use Optimized Libraries

For production workloads, use tested cryptographic libraries:

```
from Crypto.Cipher import DES3

cipher = DES3.new(key, DES3.MODE_ECB)
ciphertext = cipher.encrypt(data)
```

Libraries use:

- C implementations
- CPU optimization
- Hardware acceleration

C. Replacing DES/3DES with a Modern Cipher

Replacing DES/3DES with a modern algorithm improves both **security and performance**.

Security Comparison

Algorithm	Key Size	Block Size	Security Status
DES	56-bit	64-bit	Broken
3DES	112/168-bit	64-bit	Deprecated
AES-128	128-bit	128-bit	Secure
AES-256	256-bit	128-bit	Highly secure

Problems with DES/3DES

DES

- 56-bit key can be brute-forced.
- Vulnerable to modern computing power.

3DES

- Uses three DES operations, making it slower.
- Small 64-bit block size causes security concerns for large datasets.
- Deprecated for many new applications.

AES Advantages

1. Better Performance

AES is significantly faster because:

- Larger block size (128-bit)
- Efficient mathematical operations
- Hardware acceleration (AES-NI instructions)

Approximate comparison:

Algorithm	Relative Speed
DES	Slow
3DES	Very slow
AES-128	Fast
AES-256	Fast

---

2. Stronger Security

AES provides:

- Resistance against brute-force attacks
- Larger key sizes
- Modern cryptographic design

---

3. Better Modes of Operation

AES supports secure authenticated modes:

- AES-GCM
- AES-CCM

These provide:

- Confidentiality
- Integrity verification
- Authentication

Example:

```
from Crypto.Cipher import AES

cipher = AES.new(key, AES.MODE_GCM)

ciphertext, tag = cipher.encrypt_and_digest(data)
```

2. Performance Analysis of Block Cipher Modes of Operation
A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

Performance Analysis of Block Cipher Modes of Operation (ECB, CBC, CFB, OFB)
A Python implementation of AES-like block cipher modes can experience **high latency, excessive memory usage, and decryption failures** due to incorrect padding, improper IV handling, and inefficient processing strategies.

A. Debug Incorrect Padding and IV Handling
1. Padding Errors
Block ciphers such as AES operate on fixed-size blocks:

- AES block size = **16 bytes**
- Input data must be a multiple of 16 bytes before encryption.

A common solution is **PKCS#7 padding**.
Incorrect Padding Example
padding = b'\0' * (16 - len(data) % 16)
Problems:

- Zero padding cannot distinguish real zeros from padding.
- Decryption may remove valid data accidentally.

Correct PKCS#7 Padding

```
def pad(data, block_size=16):
    padding_length = block_size - (len(data) % block_size)

    return data + bytes([padding_length]) * padding_length
```

Example:
Input:
HELLO

Length = 5 bytes

Padding:
11 bytes of 0x0B

Result:
HELLO 0B0B0B0B0B0B0B0B0B0B0B

Correct Unpadding
Incorrect:

```
data[:-data[-1]]
```

Problem:

- Does not verify padding validity.

Correct:

```
def unpad(data):
    padding_length = data[-1]

    if padding_length < 1 or padding_length > 16:
        raise ValueError("Invalid padding")
```

```
    if data[-padding_length:] != bytes([padding_length])*padding_length:
        raise ValueError("Invalid padding")

    return data[:-padding_length]
```

---

2. IV Handling Errors
Initialization Vector (IV) problems commonly cause CBC, CFB, and OFB failures.

Common Mistakes

Error	Result
Reusing IV	Security vulnerability
Different IV during decryption	Invalid plaintext
Wrong IV length	Block mismatch
Generating IV after encryption	Decryption failure
Hard-coded IV	Predictable ciphertext

Correct IV Usage
For AES:
IV size = 16 bytes
Encryption:

```
from Crypto.Random import get_random_bytes

iv = get_random_bytes(16)

ciphertext = encrypt(data, key, iv)
```

Store:
IV + Ciphertext
Example:

```
output = iv + ciphertext
```

During decryption:

```
iv = encrypted_data[:16]
ciphertext = encrypted_data[16:]
```

B. Refactoring for Reduced Computation and Memory Usage
1. Avoid Repeated Cipher Initialization
Inefficient

```
for block in blocks:
    cipher = AES(key)
    encrypt(block)
```

Problem:

- Key expansion repeated for every block.

Optimized

```
cipher = AES(key)

for block in blocks:
    encrypt(block)
```

Benefits:

- Key schedule calculated once.
- Lower CPU usage.

---

2. Use Streaming Instead of Loading Entire Files
Inefficient

```
data = open("large_file.bin","rb").read()

cipher.encrypt(data)
```
Problems:
- High RAM usage.
- Poor performance for large files.

Optimized Streaming
```
with open("input.bin","rb") as fin:
   with open("output.bin","wb") as fout:

      while block := fin.read(16):
         encrypted = cipher.encrypt(block)
         fout.write(encrypted)
```
Advantages:
- Constant memory usage.
- Suitable for GB-sized files.

---

3. Reduce Temporary Object Creation
Inefficient
```
result = b""

for block in blocks:
   result += encrypted_block
```
Problem:
- Creates many intermediate byte objects.

Optimized
```
result = bytearray()

for block in blocks:
   result.extend(encrypted_block)
```
Advantages:
- Faster append operations.
- Lower memory overhead.

---

4. Optimize Mode-Specific Operations
ECB Mode
Encryption:

$$C_i = E(P_i)$$

Optimization:
- Encrypt blocks independently.
- Parallel processing possible.

Example:
```
cipher.encrypt(block)
```

CBC Mode
Formula:

$$C_i = E(P_i \oplus C_{i-1})$$

Optimization:
```
previous = iv

for block in blocks:
```

block = xor(block, previous)
previous = encrypt(block)

Avoid recomputing previous blocks.

CFB Mode

Formula:

$$C_i = P_i \oplus E\,(C_{i-1})$$

Optimization:

- Maintain encryption state.
- Avoid unnecessary buffers.

OFB Mode

Formula:

$$O_i = E(O_{i-1})$$

Optimization:

- Generate keystream once.
- XOR with data.

C. Comparison and Optimization for Real-Time Secure Communication

Mode Performance Comparison

Mode	Padding Required	Parallel Encryption	Parallel Decryption	Security	Speed
ECB	Yes	Yes	Yes	Poor	Fast
CBC	Yes	No	Yes	Good	Medium
CFB	No	No	No	Good	Medium
OFB	No	No	No	Good	Fast
CTR	No	Yes	Yes	Excellent	Very Fast
GCM	No	Yes	Yes	Excellent	Fast

Analysis of Each Mode

1. ECB

Advantages

- Simple implementation
- Fast processing

Problems

Identical plaintext blocks create identical ciphertext blocks.

Example:
Plaintext:

AAAA AAAA
AAAA AAAA

Ciphertext:

XXXX XXXX
XXXX XXXX
Patterns remain visible.
Not suitable for secure communication.

---

2. CBC
Advantages
- Hides plaintext patterns.
- Widely supported.

Problems
- Encryption cannot be parallelized.
- Requires padding.
- Requires unpredictable IV.

Suitable for:
- File encryption
- Legacy systems

---

3. CFB
Advantages:
- No padding required.
- Useful for streaming data.

Problems:
- Sequential processing.
- Error propagation.

Suitable for:
- Older communication protocols.

---

4. OFB
Advantages:
- Converts block cipher into stream cipher.
- No padding.
- Small error impact.

Problems:
- IV reuse completely breaks security.

Suitable for:
- Low-latency communication.

Recommended Mode for Real-Time Secure Communication
AES-GCM
Although not listed in the original implementation, AES-GCM is the preferred modern choice.
Reasons:
1. High Performance
- Parallel encryption.
- Hardware acceleration support.
- Low latency.

2. Authentication Included
Provides:

- Confidentiality
- Integrity
- Authentication tag

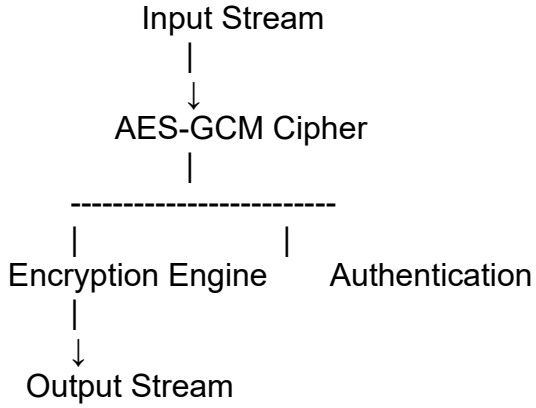
Unlike ECB/CBC/OFB, it detects modification attacks.
Example:

```
from Crypto.Cipher import AES

cipher = AES.new(key, AES.MODE_GCM)

ciphertext, tag = cipher.encrypt_and_digest(message)
```

Optimized Architecture



3. Debugging RSA Cryptosystem Implementation
A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

Debugging RSA Cryptosystem Implementation
A Python RSA implementation that fails for large messages or produces incorrect plaintext after decryption usually has problems in **prime generation, key construction, message handling, or modular arithmetic operations**.
RSA correctness depends on:

1. Selecting secure prime numbers.
2. Generating valid public/private keys.
3. Performing modular exponentiation correctly.
4. Ensuring plaintext values fit within the RSA modulus.

A. Identifying Issues in RSA Implementation
1. Prime Generation Problems
RSA security depends on two large prime numbers:

$$n = p \times q$$

where:

- $p$and $q$are large primes.
- $n$is the modulus.

Common Prime Generation Errors

Error 1: Using Small or Fixed Primes

Example:

p = 61

q = 53

Problems:

- Easily factorized.
- Provides no real security.

Fix

Generate random large primes:

```
from Crypto.Util.number import getPrime

p = getPrime(1024)
q = getPrime(1024)
```

Error 2: Weak Primality Testing

Incorrect:

```
def is_prime(n):
    for i in range(2,n):
        if n%i == 0:
            return False
```

Problems:

- Extremely slow.
- Not suitable for large numbers.

Fix: Miller-Rabin Test

Miller-Rabin provides efficient probabilistic primality checking.

```
def is_probable_prime(n, rounds=40):
    # Miller-Rabin implementation
    pass
```

Advantages:

- Fast for 2048-bit numbers.
- High confidence of correctness.

---

2. Key Generation Errors

RSA key generation:

$$\phi(n) = (p-1)(q-1)$$

Choose public exponent:

$$gcd(e, \phi(n)) = 1$$

Private key:

$$d = e^{-1} \bmod \phi(n)$$

Common Errors

Error 1: Invalid Public Exponent

Incorrect:

e = 100

Problem:

- May not be relatively prime to φ(n).

Correct:
e = 65537
65537 is widely used because:

- Efficient exponentiation.
- Good security properties.

Error 2: Incorrect Private Key Calculation
Incorrect:
d = 1/e
Problem:

- Normal division is not valid.

Correct:
d = pow(e, -1, phi)

Error 3: Incorrect φ(n)
Incorrect:
phi = p*q
Correct:
phi = (p-1)*(q-1)
Using the wrong Euler function causes decryption failure.

---

3. Modular Exponentiation Problems
RSA encryption:

$$C = M^e \mod n$$

Decryption:

$$M = C^d \mod n$$

Common Error
Using normal exponentiation:
ciphertext = message ** e % n
Problem:

- Creates extremely large intermediate values.
- Causes memory and speed issues.

Fix: Fast Modular Exponentiation
Use Python's optimized modular power:
ciphertext = pow(message, e, n)

plaintext = pow(ciphertext, d, n)
Advantages:

- Uses square-and-multiply algorithm.
- Avoids huge intermediate numbers.

---

4. Large Message Encryption Failure
RSA cannot directly encrypt arbitrary large messages.
Condition:

$$M < n$$

If:
message > n

encryption fails.

Incorrect Approach
cipher = pow(message,e,n)
with a 10 MB file.
Problems:

- RSA is not designed for bulk data.
- Very slow.
- Message exceeds modulus size.

Correct Solution: Hybrid Encryption
Use RSA only for key exchange:
Large Data
|
↓
AES Encryption
|
↓
Encrypted AES Key
|
↓
RSA Encryption
Example:
RSA:
Encrypt AES session key

AES:
Encrypt actual message
This is the approach used in systems such as TLS.

B. Optimization Techniques
1. Fast Modular Exponentiation
The square-and-multiply algorithm reduces complexity:
Normal exponentiation:

$$O(e)$$

Fast exponentiation:

$$O\big(log(e)\big)$$

Example:

```
def mod_exp(base, exponent, modulus):

    result = 1

    while exponent > 0:

        if exponent % 2:
            result = (result * base) % modulus

        base = (base * base) % modulus
        exponent //= 2

    return result
```

---

2. Chinese Remainder Theorem (CRT)
RSA decryption:

$$M = C^d \bmod n$$

can be optimized using CRT.
Instead of calculating:

$$mod\ n$$

calculate:

$$mod\ p$$

and:

$$mod\ q$$

separately.

Precompute:

$$d_p = d \bmod (p-1)$$
$$d_q = d \bmod (q-1)$$
$$q^{-1} \bmod p$$

CRT decryption:

```
def decrypt_CRT(ciphertext,p,q,dp,dq,qinv):

    m1 = pow(ciphertext, dp, p)
    m2 = pow(ciphertext, dq, q)

    h = (qinv*(m1-m2)) % p

    return m2 + h*q
```

CRT Advantages

Method	**Operations**
Normal RSA	One large exponentiation
CRT RSA	Two smaller exponentiations

Performance improvement:

- Approximately **3–4× faster RSA decryption**.

---

3. Efficient Key Storage
Instead of storing:
(p,q,n,e,d)
store CRT parameters:
(
p,
q,
dp,
dq,
qinv
)
Benefits:

- Faster decryption.
- Less computation.

---

4. Use Optimized Cryptographic Libraries
Pure Python RSA is slow.
Recommended:
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
Benefits:

- Native optimized arithmetic.
- Secure padding.
- Side-channel protections.

C. Trade-off Between RSA Key Size, Security, and Performance
RSA Key Size Comparison

RSA Key Size	Security Level	Performance
1024-bit	Weak/deprecated	Fast
2048-bit	Standard security	Balanced
3072-bit	Higher security	Slower
4096-bit	Very high security	Slowest

---

1. Effect on Encryption Speed
RSA encryption uses:

$$C = M^e \mod n$$

Usually:

- Public exponent $e = 65537$
- Encryption is relatively fast.

---

2. Effect on Decryption Speed
Private exponent:

$$M = C^d \mod n$$

is much larger.
Therefore:

- Decryption is slower.
- CRT optimization becomes important.

---

3. Security vs Performance Trade-off

Requirement	Recommended Configuration
Embedded systems	RSA-2048
General applications	RSA-2048 + OAEP
Long-term protection	RSA-3072/4096
High-volume encryption	RSA + AES hybrid

---

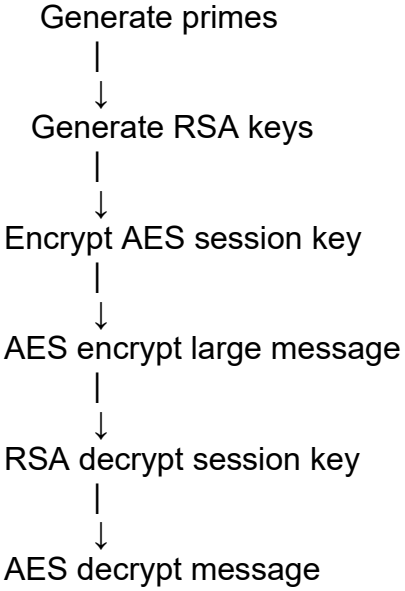
4. RSA vs Modern Alternatives
RSA is still widely used, but newer systems increasingly use:

Algorithm	Advantage
RSA-2048	Compatibility
ECC	Smaller keys, faster operations
AES	Bulk data encryption

Example:
RSA-2048 security is roughly comparable to:

- ECC-224/256-bit security.

Optimized RSA Workflow



---

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange
A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

Optimization and Security Analysis of Diffie-Hellman Key Exchange
A Python Diffie-Hellman (DH) implementation may generate incorrect shared keys or suffer from poor performance because of improper modular arithmetic, inefficient exponentiation, weak parameter selection, and lack of authentication. The following analysis addresses debugging, optimization, and security enhancement.

A. Debug Incorrect Shared Key Generation
1. Diffie-Hellman Operation Overview
Diffie-Hellman uses:

- A large prime number p
- A generator g
- Alice private key a
- Bob private key b

Public keys:

$$A = g^a \bmod p$$
$$B = g^b \bmod p$$

Shared secret:
Alice computes:

$$K = B^a \bmod p$$

Bob computes:

$$K = A^b \bmod p$$

Both obtain:

$$K = g^{ab} \mod p$$

---

2. Common Implementation Errors
Error 1: Incorrect Modular Exponentiation
Wrong implementation
A = (g ** private_key) % p
Problem

- Generates extremely large intermediate values.
- Causes slow execution and memory consumption.

Correct implementation
A = pow(g, private_key, p)
Python performs efficient modular exponentiation internally.

---

3. Incorrect Shared Key Calculation
Incorrect
shared_secret = B * private_key % p
Problem:

- Performs multiplication instead of exponentiation.

Correct
shared_secret = pow(B, private_key, p)

---

4. Parameter Mismatch
Both parties must use identical:

- Prime number p
- Generator g

Incorrect:
Alice:
p = 23
g = 5

Bob:
p = 29
g = 5
Result:

- Different shared keys.
- Decryption failure in later encryption stages.

Correct:
p = shared_prime
g = generator

---

5. Weak Private Key Generation
Incorrect:
private_key = 10
Problem:

- Predictable private value.
- Allows attackers to calculate the secret.

Correct:
import secrets

private_key = secrets.randbelow(p-2) + 1

Corrected Python Example

```
import secrets

p = 23
g = 5

# Alice
a = secrets.randbelow(p-2) + 1
A = pow(g, a, p)

# Bob
b = secrets.randbelow(p-2) + 1
B = pow(g, b, p)

# Shared secret
alice_secret = pow(B, a, p)
bob_secret = pow(A, b, p)

if alice_secret == bob_secret:
    print("Shared key generated successfully")
else:
    print("Key mismatch")
```

B. Optimization of Exponentiation for Large Prime Values

Large DH systems typically use:

- 2048-bit primes
- 3072-bit primes
- 4096-bit primes

Naive exponentiation becomes inefficient.

---

1. Use Fast Modular Exponentiation

Inefficient

```
result = (base ** exponent) % modulus
```

Problems:

- Creates huge numbers before applying modulus.
- High memory usage.

Optimized

```
result = pow(base, exponent, modulus)
```

Advantages:

- Uses square-and-multiply algorithm.
- Avoids large intermediate values.
- Reduces execution time.

---

2. Square-and-Multiply Algorithm

The algorithm reduces exponentiation complexity.

Normal approach:

$$O(n)$$

Optimized approach:

$$O(log(n))$$

Implementation:

```
def modular_exponentiation(base, exponent, modulus):
```

```
    result = 1
    base = base % modulus

    while exponent > 0:

        if exponent & 1:
            result = (result * base) % modulus

        base = (base * base) % modulus
        exponent >>= 1

    return result
```

---

3. Use Secure Predefined DH Groups

Generating large primes repeatedly is expensive.
Instead of:
generate_prime(2048)
for every session:
Use standardized safe groups.
Benefits:

- Faster connection setup.
- Avoids weak primes.
- Provides tested security parameters.

---

4. Use Safe Prime Generation

A secure DH prime follows:

$$p = 2q + 1$$

where $q$is also prime.
Advantages:

- Prevents small subgroup attacks.
- Improves resistance against cryptanalysis.

---

5. Reduce Repeated Computation

Inefficient:

```
for connection in users:
    generate_prime()
```

Optimized:

```
p = predefined_safe_prime

for connection in users:
    generate_private_key()
```

Benefits:

- Faster key exchange.
- Lower CPU usage.

C. Preventing Man-in-the-Middle Attacks

1. Problem with Basic Diffie-Hellman

Traditional DH provides secrecy but no identity verification.
An attacker can intercept:

```
Alice           Attacker          Bob

 A ---------->     X
```

fake_A --------->

fake_B <----------

The attacker establishes two separate keys:

- Alice ↔ Attacker
- Attacker ↔ Bob

This is a Man-in-the-Middle (MITM) attack.

---

2. Add Digital Signatures

Each participant signs their DH public value.

Process:

Alice:

$$Signature = Sign(A)$$

Bob:

$$Verify(S, ignatureA)$$

Only valid keys are accepted.

Example:

signature = sign(private_signing_key, A)

Verification:

verify(public_signing_key, A, signature)

Benefits:

- Confirms identity.
- Prevents key substitution attacks.

---

3. Use Certificates

A practical system uses:

- Public key certificates
- Certificate authorities
- Identity verification

Example:

Certificate

|

↓

Verify public key

|

↓

Accept DH exchange

This is the approach used in secure protocols such as TLS.

---

4. Apply Key Derivation Functions (KDF)

The raw DH output should not directly become an encryption key.

Incorrect:

```
AES_key = shared_secret
```

Correct:

```
from hashlib import sha256

AES_key = sha256(
   str(shared_secret).encode()
).digest()
```

Benefits:

- Generates fixed-length keys.
- Removes mathematical structure.
- Improves security.

---

5. Add Nonces for Replay Protection
Example:
nonce = secrets.token_bytes(16)
Used with:
- Session identifiers
- Authentication tags
- Timestamps

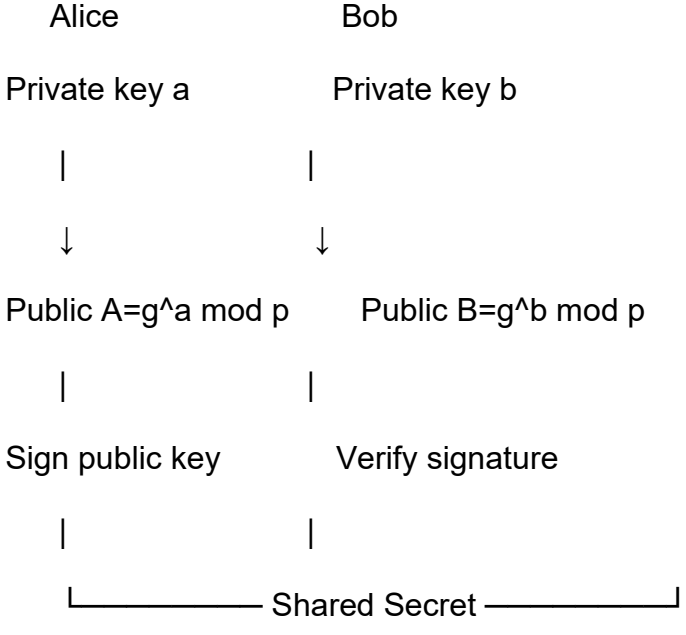
Performance Impact of Security Improvements

Enhancement	Security Improvement	Performance Impact
Fast modular exponentiation	Faster DH calculation	Improves speed
Safe primes	Prevents subgroup attacks	Small overhead
Larger key sizes	Stronger security	Higher computation
Digital signatures	Prevents MITM	Additional CPU cost
KDF	Secure session keys	Minimal overhead
Nonces	Prevents replay attacks	Very small overhead

Key Size vs Performance Comparison

DH Key Size	Security Level	Performance
1024-bit	Weak	Fast
2048-bit	Recommended	Balanced
3072-bit	Strong	Slower
4096-bit	Very strong	Highest cost

Optimized Secure DH Architecture



|

↓

KDF

|

↓

AES Session Key

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation
A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation
An ECC implementation can produce incorrect key exchange results due to errors in elliptic curve arithmetic, incorrect handling of the point at infinity, invalid modular operations, or inefficient scalar multiplication. Proper implementation of point operations is essential because ECC security depends on accurate finite-field mathematics.

A. Identify and Fix Errors in Point Addition and Scalar Multiplication Logic
1. Elliptic Curve Fundamentals
ECC commonly uses curves of the form:

$$y^2 = x^3 + ax + b \pmod{p}$$

where:

- $p$is a large prime defining the finite field.
- $a$and $b$define the curve.
- A point on the curve is:

$$P = (x, y)$$

The point at infinity Oacts as the identity element.

---

2. Common Point Addition Errors
Point addition calculates:

$$R = P + Q$$

where $P$and $Q$are curve points.

Error 1: Incorrect Handling of the Point at Infinity
Incorrect:

```
if P is None:
    return Q
```

Problem:

- Does not handle all identity cases.

Correct:

```
def point_add(P, Q):

    if P is None:
        return Q

    if Q is None:
        return P
```

Error 2: Incorrect Inverse Point Handling

For:

$$P = (x, y)$$

the inverse is:

$$-P = (x, -y \bmod p)$$

If:

$$P + (-P) = O$$

the implementation must return infinity.

Incorrect:

```
if P[0] == Q[0]:
    return None
```

Problem:

- Incorrectly treats all equal x-coordinates as infinity.

Correct:

```
if P[0] == Q[0] and (P[1]+Q[1]) % p == 0:
    return None
```

---

3. Correct Point Addition Formula

Case 1: Different Points

For:

$$P \neq Q$$

Slope:

$$m = \frac{y_2 - y_1}{x_2 - x_1} \bmod p$$

Coordinates:

$$x_3 = m^2 - x_1 - x_2$$
$$y_3 = m(x_1 - x_3) - y_1$$

Case 2: Point Doubling

For:

$$P = Q$$

Slope:

$$m = \frac{3x_1^2 + a}{2y_1} \bmod p$$

Correct Python Implementation

```
def inverse_mod(k, p):
    return pow(k, -1, p)
```

```
def point_add(P, Q, a, p):

    if P is None:
        return Q

    if Q is None:
        return P


    x1, y1 = P
    x2, y2 = Q


    # P + (-P) = O
    if x1 == x2 and (y1 + y2) % p == 0:
        return None


    # Point doubling
    if P == Q:

        numerator = (3*x1*x1 + a) % p
        denominator = inverse_mod(2*y1, p)

        m = (numerator * denominator) % p


    # Normal addition
    else:

        numerator = (y2-y1) % p
        denominator = inverse_mod(x2-x1, p)

        m = (numerator * denominator) % p


    x3 = (m*m - x1 - x2) % p
    y3 = (m*(x1-x3)-y1) % p


    return (x3, y3)
```

---

4. Scalar Multiplication Errors
Scalar multiplication computes:

$$kP = P + P + \ldots + P$$

for $k$times.

Incorrect Method

```
result = None

for i in range(k):
    result = point_add(result, P)
```

Problems:

- Extremely slow.
- Complexity:

$$O(k)$$

B. Optimization Using Double-and-Add

1. Double-and-Add Algorithm

The binary representation of $k$is used.

Example:

$$13 = (1101)_2$$

Instead of adding Pthirteen times:

$$13P = 8P + 4P + P$$

Optimized Algorithm

```
def scalar_multiply(k, P, a, p):

    result = None
    current = P


    while k > 0:

        if k & 1:
            result = point_add(
                result,
                current,
                a,
                p
            )

        current = point_add(
            current,
            current,
            a,
            p
        )

        k >>= 1


    return result
```

Performance Improvement

Comparison:

Method	**Complexity**
Repeated addition	O(k)
Double-and-add	O(log k)

Example:

For a 256-bit private key:

Repeated addition:

$$2^{256}$$

operations possible.
Double-and-add:

$$256$$

iterations approximately.

---

2. ECC Key Exchange Example
Elliptic Curve Diffie-Hellman (ECDH):
Public Parameters
Curve:

$$y^2 = x^3 + ax + b$$

Generator point:

$$G$$

Alice
Private key:

$$d_A$$

Public key:

$$Q_A = d_A G$$

Bob
Private key:

$$d_B$$

Public key:

$$Q_B = d_B G$$

Shared Secret
Alice:

$$S = d_A Q_B$$

Bob:

$$S = d_B Q_A$$

Both produce:

$$S = d_A d_B G$$

Python example:

```
alice_private = 7
bob_private = 11


alice_public = scalar_multiply(
    alice_private,
    G,
    a,
    p
```

```
)


bob_public = scalar_multiply(
    bob_private,
    G,
    a,
    p
)


alice_secret = scalar_multiply(
    alice_private,
    bob_public,
    a,
    p
)


bob_secret = scalar_multiply(
    bob_private,
    alice_public,
    a,
    p
)


assert alice_secret == bob_secret
```

---

3. Additional ECC Optimizations

a. Use Jacobian Coordinates

Standard affine coordinates require modular inversion:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

Modular inversion is expensive.
Jacobian coordinates represent:

$$(x, y)$$

as:

$$(X, Y, Z)$$

Benefits:

- Avoid frequent inversions.
- Faster point multiplication.

---

b. Use Windowed Scalar Multiplication

Instead of one bit at a time:

- Process multiple bits together.
- Precompute points.

Advantages:

- Faster repeated operations.
- Useful in high-performance ECC systems.

---

c. Use Optimized Libraries

Production ECC should use tested libraries:
from cryptography.hazmat.primitives.asymmetric import ec
Advantages:

- Constant-time implementations.
- Side-channel resistance.
- Standard curves.

Security Checks

1. Validate Points

Before using a received point:
Check:

$$y^2 \mod p = (x^3 + ax + b) \mod p$$

Example:

```
def validate_point(P,a,b,p):

    x,y=P

    return (
        y*y % p ==
        (x*x*x+a*x+b)%p
    )
```

---

2. Use Standard Curves

Avoid custom curves unless mathematically verified.
Common secure curves:

- NIST P-256
- Curve25519
- Ed25519

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation